

Object oriented Programming | SAQ

1. Explain why method Overloading improves the readability of a Java program.

Method Overloading and Readability

Method overloading improves the readability of a Java program by allowing multiple methods with the same name to perform similar tasks with different parameters.

Instead of using different names for similar operations, we can use one meaningful method name.

Example:

```
void add(int a, int b)
void add(int a, int b, int c)
```

Here, the same `add()` name makes the program simple, clear, and easy to understand.

2. Explain the conditional operator i) with an example,

Conditional Operator

The conditional operator (`?:`) is a ternary operator used to make a decision between two values. It is a short form of the if-else statement.

Syntax

```
condition ? value1 : value2;
```

Example

```
int age = 20;
String result = (age ≥ 18) ? "Adult" : "Minor";
System.out.println(result);
```

Output:

```
Adult
```

Here, if `age ≥ 18` is true, `"Adult"` is selected; otherwise, `"Minor"` is selected.

3. Explain the difference between calling a method using an object and calling a static method.

Difference Between Object Method and Static Method

An object method is a method that is called using an object, while a static method belongs to the class and is called using the class name.

Object Method	Static Method
Declared without <code>static</code> .	Declared with <code>static</code> .
Called using an object.	Called using the class name.
Can directly access instance variables.	Cannot directly access instance variables.
Each object can use the method.	One common method is shared by the class.

Example

```
class Test {  
    void show() {  
        System.out.println("Object Method");  
    }  
  
    static void display() {  
        System.out.println("Static Method");  
    }  
  
    public static void main(String[] args) {  
        Test t = new Test();  
        t.show(); // Object method  
        Test.display(); // Static method  
    }  
}
```

In short: Object methods are called through an object, whereas static methods are called through the class name.

4. Differentiate between standard library methods and user-defined methods in Java.

Standard Library Methods	User-Defined Methods
Already provided by Java.	Created by the programmer.
Can be used directly by importing the required class/package.	Must be defined by the programmer before use.
Used for common tasks.	Used for specific tasks required by the program.
Example: <code>Math.sqrt()</code>	Example: <code>calculateSum()</code>

Example

```
Math.sqrt(25); // Standard library method

int calculateSum(int a, int b) {
    return a + b; // User-defined method
}
```

In short: Standard library methods are predefined by Java, while user-defined methods are created by the programmer.

5. Design a simple class for a "Library Book" with appropriate properties and methods.

Design a simple class for a “Library Book” with appropriate properties and methods.

Answer

```
class LibraryBook {
    // Properties
    String title;
    String author;
    int bookId;

    // Methods
    void issueBook() {
        System.out.println("Book issued.");
    }

    void returnBook() {
        System.out.println("Book returned.");
    }

    void display() {
        System.out.println("Title: " + title);
        System.out.println("Author: " + author);
        System.out.println("Book ID: " + bookId);
    }
}
```

Properties

- `title` → Book title
- `author` → Author name

- `bookId` → Unique book ID

Methods

- `issueBook()` → Issues the book
- `returnBook()` → Returns the book
- `display()` → Displays book information

7. propose how polymorphism Can be applied developing a banking system

Polymorphism in a Banking System

Polymorphism can be applied in a banking system by using the same method name with different implementations for different types of bank accounts.

For example, `calculateInterest()` can behave differently for Savings Account and Current Account.

```
class Account {
    void calculateInterest() {
        System.out.println("General interest");
    }
}

class SavingsAccount extends Account {
    void calculateInterest() {
        System.out.println("Savings account interest");
    }
}

class CurrentAccount extends Account {
    void calculateInterest() {
        System.out.println("Current account interest");
    }
}
```

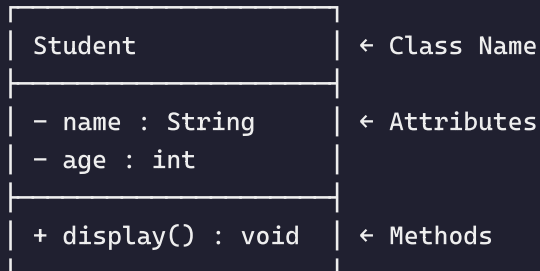
Here, the same `calculateInterest()` method performs different tasks depending on the account type. This makes the banking system flexible and easier to maintain.

8. Explain the concept of UML (Unified Modeling Language) and how it represents a class.

UML is a standard visual language used to design and represent a software system. It uses diagrams to show the structure, behavior, and relationships of a system.

Representation of a Class

A UML class is usually represented by a rectangle divided into three sections:



- **Top:** Class name
- **Middle:** Attributes/properties
- **Bottom:** Methods/operations

In short: UML provides a visual way to model classes and their relationships before implementing them in Java.

9. Describe the syntax for creating an object in Java.

Syntax for Creating an Object in Java

The general syntax for creating an object is:

```
ClassName objectName = new ClassName();
```

Example

```
Student s1 = new Student();
```

10. Provide an example Of an Object and list its States (properties) and behaviors (functions).

Example of an Object

Object: Car

States / Properties

- Color
- Brand
- Speed
- Model

Behaviors / Functions

- `start()`
- `stop()`
- `accelerate()`
- `brake()`

Example: A Car object has states such as color and speed, and behaviors such as starting, stopping, and accelerating.

11. Explain the difference between break and continue

Difference Between `break` and `continue`

`break`

Terminates the loop completely.

Used when we want to exit from a loop.

`continue`

Skips the current iteration and continues with the next iteration.

Used when we want to skip a particular condition.

Example

```
for (int i = 1; i ≤ 5; i++) {  
    if (i == 3)  
        break;  
    System.out.println(i);  
}
```

Output:

```
1  
2
```

```
for (int i = 1; i ≤ 5; i++) {  
    if (i == 3)  
        continue;  
    System.out.println(i);  
}
```

Output:

1
2
4
5